第 5 章：集合与字典

本章目标：不只会写 `set` 和 `dict`，还要能根据“唯一成员关系”与“键值映射”选择数据结构，并判断一次操作会创建新容器还是修改原容器。配套代码：`example.py` / `Chapter05SetsDicts.java`。

建议先阅读本章，再直接运行：

```
python3 java_to_python/chapters/05_sets_dicts/example.py
```

示例输出按照 5.1～5.12 分段。学习时先猜输出，再替换技能、权限和商品数据重跑。

学习目标

学完本章后，你应该能够：

- 用 `set` 表达“有哪些唯一成员”，用 `dict` 表达“一个键对应一个值”；
- 解释为什么集合元素和字典键必须可哈希；
- 正确选择 `add` / `update`、`remove` / `discard` 和各种集合运算；
- 区分 `mapping[key]`、`get`、`setdefault` 与 `defaultdict` 的行为；
- 理解集合运算、字典合并、别名和原地修改之间的关系；
- 安全遍历字典的动态视图，并避免在遍历时改变字典大小；
- 使用 `Counter` 和 `defaultdict` 完成计数与分组；
- 识别可变默认值共享、缺失键和无可靠集合顺序等常见问题。

5.1 两种“按内容查找”的容器

列表把数据放在有顺序的位置中；集合关心某个唯一成员是否存在；字典则通过唯一的键找到值。可以先用下面三个问题选择容器：

问题	合适的类型	Java 心智模型
第几个位置是什么？	`list / tuple`	`List<T>`
某个唯一成员在不在？	`set`	`HashSet<T>`

5.1 两种“按内容查找”的容器

列表把数据放在有顺序的位置中；集合关心某个唯一成员是否存在；字典则通过唯一的键找到值。
可以先用下面三个问题选择容器：

问题	合适的类型	Java 心智模型
第几个位置是什么？	`list / tuple`	`List<T>`
某个唯一成员在不在？	`set`	`HashSet<T>`
某个键对应什么值？	`dict`	`HashMap<K, V>`

```
languages = ["Java", "Python", "Java"]
unique_languages = {"Java", "Python"}
scores = {"Alice": 95, "Bob": 88}

print(languages[0])                  # Java：按位置读取
print("Python" in unique_languages)  # True：判断成员
print(scores["Alice"])               # 95：通过键读取值
```

Python 容器保存的是对象引用。把一个集合或字典赋给另一个变量不会自动复制对象；只要两个名字指向同一可变容器，通过任一名字进行原地修改，另一边都能观察到。

5.2 创建集合：唯一、无可靠顺序

集合字面量使用花括号，重复元素只保留一份：

```
skills = {"Java", "Python", "Java"}
print(len(skills))          # 2
print(sorted(skills))       # ['Java', 'Python']
```

`set(iterable)` 会遍历输入并收集唯一元素，因此常用于去重：

```
raw_tags = ["backend", "python", "backend", "api"]
unique_tags = set(raw_tags)
print(sorted(unique_tags))  # ['api', 'backend', 'python']
```

这里使用 `sorted` 只是为了让展示结果稳定。不要依赖集合的打印或迭代顺序；即使某次运行看起来固定，也不代表它具有可依赖的业务含义。

空集合的特殊写法

```
empty_set = set()
empty_dict = {}

print(type(empty_set).__name__)   # set
print(type(empty_dict).__name__)  # dict
```

`{}` 已经表示空字典，所以空集合必须写成 `set()`。这是本章最容易出现的字面量错误之一。

什么是“可哈希”

集合需要快速判断一个元素是否已经存在。Python 会先计算元素的哈希值（hash），把它缩小到可能存放的位置，再用相等性判断确认。因此，集合元素必须是**可哈希对象**：在参与查找期间，它的哈希值和相等性不能随内部内容任意改变。

常见情况如下：

对象	通常能否作为集合元素或字典键	原因
`int` 、 `float` 、 `str` 、 `bytes`	能	这些对象不可变
只包含可哈希元素的 `tuple`	能	元组及其内部元素都满足要求
`list` 、 `dict` 、 `set`	不能	内容可原地改变
`frozenset`	能	它是不可变集合

```
owners = {("service-a", "Alice"), ("service-b", "Bob")}

try:
    invalid = {["Java", "Python"]}
except TypeError as error:
    print(type(error).__name__, error)  # unhashable type: 'list'
```

这段异常捕获只是为了观察教学反例。实际代码应在设计数据结构时就选用合适类型，而不是每次添加元素时再捕获 `TypeError`。元组也不是无条件可哈希：`("Alice", ["admin"])` 内部仍有

5.3 集合适合成员判断与去重

集合最直观的用途是表达允许名单、已处理编号、标签和权限：

```
allowed_roles = {"admin", "operator"}
role = "operator"

if role in allowed_roles:
    print("允许操作")
```

`in` 在集合中通常能直接根据哈希值定位，而列表需要从前向后比较。数据量较大且需要反复判断成员时，集合通常比列表更合适；小数据场景仍应优先考虑语义是否清楚，不必为了微小性能差异把所有列表都改成集合。

去重时是否需要保留顺序

```
values = ["Java", "Python", "Java", "Go"]

unordered_unique = set(values)
ordered_unique = list(dict.fromkeys(values))

print(sorted(unordered_unique))  # 只关心唯一值
print(ordered_unique)            # ['Java', 'Python', 'Go']
```

如果只关心有哪些不同值，用集合；如果还要保留第一次出现的顺序，可以利用字典键唯一且保留插入顺序的性质。不要把 `list(set(values))` 当成稳定的顺序去重方案。

5.4 修改集合

集合是可变对象。常用方法可以按“添加”和“删除”来记：

方法	行为	关键边界
`add(value)`	添加一个完整元素	已存在时不重复添加
`update(iterable)`	遍历输入并添加多个元素	字符串会被拆成字符
`remove(value)`	删除指定元素	不存在时抛 `KeyError`
`discard(value)`	删除指定元素	不存在时安静忽略

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

`discard(value)`	删除指定元素	不存在时安静忽略
`pop()`	删除并返回任意元素	不是“最后一个”；空集合抛 `KeyError`
`clear()`	删除全部元素	原地修改并返回 `None`

```
skills = {"Java"}
skills.add("Python")
skills.update(["SQL", "Git"])
print(sorted(skills))

skills.discard("Rust")  # 不存在也没关系

try:
    skills.remove("Rust")
except KeyError:
    print("remove 认为缺失元素是错误")
```

`add` 把实参当作一个元素，`update` 把实参当作一批可迭代数据。下面是常见误区：

```
letters = {"Java"}
letters.update("SQL")
print(sorted(letters))  # ['Java', 'L', 'Q', 'S']
```

想添加完整字符串时写 `add("SQL")`；想添加多个字符串时写 `update(["SQL", "Git"])`。

与列表的原地方法类似，`add`、`update`、`discard` 和 `clear` 都返回 `None`。不要写 `skills = skills.add("Python")`，否则变量会重新绑定到 `None`。

5.5 集合代数与关系判断

当数据表达权限、标签、共同好友或已完成任务时，集合运算通常比嵌套循环更贴近问题本身：

意图	运算符	方法形式	Java `Set` 常见做法
并集	`a | b`	`a.union(b)`	复制后 `addAll`
交集	`a & b`	`a.intersection(b)`	复制后 `retainAll`
差集	`a - b`	`a.difference(b)`	复制后 `removeAll`

对称差	`a ^ b`	`a.symmetric_difference(b)`	组合两个方向的差集
子集	`a <= b`	`a.issubset(b)`	`b.containsAll(a)`
真子集	`a < b`	无需特殊写法	包含且不相等
是否无交集	—	`a.isdisjoint(b)`	遍历判断

```
backend = {"Java", "Python", "SQL"}
data = {"Python", "SQL", "Pandas"}

print(sorted(backend | data))  # 并集
print(sorted(backend & data))  # 交集
print(sorted(backend - data))  # 只在 backend 中
print(sorted(backend ^ data))  # 只在其中一边
```

关系运算同样很适合权限校验：

```
granted = {"order:read", "order:write", "report:read"}
required = {"order:read", "order:write"}

print(required <= granted)       # True：所需权限是已有权限的子集
print(required - granted)        # set()：没有缺失权限
```

新集合与原地修改

`a | b`、`a & b`、`a - b` 和 `a ^ b` 都创建新集合，不修改两侧输入。带 `=` 的版本会原地修改左侧集合：

```
current = {"read"}
alias = current

combined = current | {"write"}
print(alias)             # {'read'}：原集合没变

current |= {"delete"}
print(alias)             # alias 也能看到 delete
print(current is alias)  # True
```

z72502x6dg8tpjj0st140000gp/T/crossnote-f-6789185d35a23d3e67dca153f08.html

... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商

如果调用者仍持有左侧集合，是否使用原地运算就是可观察行为。拿不准时，明确创建新集合通常更容易控制副作用。

5.6 `frozenset`：不可变集合

`frozenset` 支持成员判断和集合代数，但没有 `add`、`remove` 等修改方法。由于内容创建后不变，它本身可以成为字典键或另一个集合的元素：

```
role_by_permissions = {
    frozenset({"read"}): "viewer",
    frozenset({"read", "write"}): "editor",
}

current_permissions = frozenset({"write", "read"})
print(role_by_permissions[current_permissions])  # editor
```

普通 `set` 适合“这组成员之后还会变化”；`frozenset` 适合“这组成员本身要作为一个稳定值”。它只冻结集合结构，并不会神奇地把任意不可哈希对象变得可哈希。

5.7 创建字典：唯一键对应一个值

字典可以想成一排带唯一标签的抽屉：键是标签，值是抽屉里的对象。字面量写作 `键: 值`：

```
scores: dict[str, int] = {"Alice": 95, "Bob": 88}
scores["Cara"] = 91

print(scores)  # {'Alice': 95, 'Bob': 88, 'Cara': 91}
```

还可以从键值对构造：

```
pairs = [("host", "localhost"), ("port", 8080)]
config = dict(pairs)
print(config["port"])  # 8080
```

字典键与集合元素遵守相同的可哈希规则；值可以是列表、集合、其他字典等任意对象。一个键只能对应一个当前值，再次赋值会覆盖旧值：

```
scores["Alice"] = 96
print(scores["Alice"])  # 96
```

现代 Python 字典保留插入顺序。更新已有键不会把它移到末尾；删除后重新插入才会成为最新的键。保留插入顺序不等于按键自动排序，需要排序时仍应显式使用 `sorted` 。

5.8 字典取值：缺失键是不是错误

取值方式没有绝对优劣，关键是先决定“缺少这个键”在业务上代表什么：

写法	键存在时	键不存在时	是否插入默认值
`mapping[key]`	返回值	抛 `KeyError`	否
`mapping.get(key)`	返回值	返回 `None`	否
`mapping.get(key, default)`	返回值	返回 `default`	否
`mapping.setdefault(key, default)`	返回当前值	插入并返回 `default`	是

```
config = {"timeout": 30, "proxy": None}

print(config["timeout"])          # 30
print(config.get("retries", 3))   # 3
print("retries" in config)        # False: get 没有插入默认值
```

如果缺键说明配置损坏，使用 `mapping[key]` 让错误尽早暴露；如果缺键是正常情况，使用 `get` 表达回退规则。不要无条件用 `get` 把拼写错误也变成 `None` 。

`None` 可能是合法值

```
print(config.get("proxy"))           # None: 键存在，值为 None
print(config.get("missing_proxy"))   # None: 键不存在

print("proxy" in config)             # True
print("missing_proxy" in config)     # False
```

当 `None` 本身有业务含义时，先用 `key in mapping` 区分“存在但值为空”和“键根本不存在”。`value in mapping` 默认检查的是键；检查值需要显式写 `value in mapping.values()` 。

2502x6dg6rpjj0sn140000gp/T/crosshote-1-6789185d35a23d3e67dca153f08.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单i

函数调用的实参会在调用前求值，所以 `get(key, build_default())` 和 `setdefault(key, build_default())` 即使最终用不到默认值，也会先执行 `build_default()` 。
默认值构造很昂贵或带副作用时，应先做显式成员判断，或使用合适的延迟创建逻辑。

5.9 修改、合并与删除字典

给不存在的键赋值会新增条目，给已有键赋值会覆盖值：

```
profile = {"name": "Alice", "city": "Shanghai"}
profile["level"] = 3
profile["city"] = "Hangzhou"
```

`update` 和 `|=` 原地合并，右侧的同名键覆盖左侧； `|` 创建一个新字典：

```
defaults = {"theme": "light", "page_size": 20}
user_settings = {"theme": "dark"}

effective = defaults | user_settings
print(effective)  # {'theme': 'dark', 'page_size': 20}
print(defaults)   # 原字典不变

defaults.update({"language": "zh-CN"})
defaults |= {"page_size": 50}
```

常用删除方式如下：

写法	返回值	缺键行为
`del mapping[key]`	无	抛 `KeyError`
`mapping.pop(key)`	被删的值	抛 `KeyError`
`mapping.pop(key, default)`	被删的值或默认值	返回默认值
`mapping.popitem()`	最后插入的 `(key, value)`	空字典抛 `KeyError`
`mapping.clear()`	`None`	原地清空

`popitem()` 的“最后插入”依赖字典顺序，但它不是“按最大键删除”，也不能传入指定键。

和集合一样，简单赋值只会创建别名：

2502x6dg8fpjj0sff40000gp/T/crossnote-f-6789185d35a23d3e67dca153f08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

和集合一样，简单赋值只会创建别名：

```
original = {"status": "pending"}
alias = original
copied = original.copy()

original["status"] = "paid"
print(alias["status"])   # paid: 同一个字典
print(copied["status"])  # pending: 不同的外层字典
```

`copy()` 是浅拷贝。如果值是列表或字典，新旧外层字典仍会共享内部可变对象；这与第 4 章的列表浅拷贝遵循同一规则。

5.10 遍历字典与动态视图

直接遍历字典得到键；需要值或键值对时分别使用 `values()` 和 `items()`：

```
scores = {"Alice": 95, "Bob": 88}

for name in scores:
    print(name)

for name, score in scores.items():
    print(name, score)
```

`keys()`、`values()` 和 `items()` 返回的是动态视图，不是当时内容的列表副本：

```
names = scores.keys()
scores["Cara"] = 91
print(list(names))  # ['Alice', 'Bob', 'Cara']
```

视图会反映后续修改，而且通常不复制整份数据。如果确实需要某一刻的快照，再显式转换成 `list(...)` 或其他合适容器。

不要在遍历时改变字典大小

```
# 教学反例：运行时会抛 RuntimeError
# for name in scores:
```

72502x6dg8fpjj0sff40000gp/T/crossnote-f-6789185d35a23d3e67dca153f08.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计量

不要在遍历时改变字典大小

```
# 教学反例：运行时会抛 RuntimeError
# for name in scores:
#     if scores[name] < 90:
#         del scores[name]
```

遍历过程中新增或删除键会改变字典大小，使当前迭代器失效。可以先复制键，再修改原字典：

```
for name in list(scores):
    if scores[name] < 90:
        del scores[name]
```

也可以构造一个只包含目标条目的新字典。选择标准仍然是：调用者是否应该观察到原对象被修改。

5.11 计数与分组

使用普通字典计数

每次遇到一个单词，就把当前次数取出并加一：

```
words = ["apple", "banana", "apple"]
counts: dict[str, int] = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)  # {'apple': 2, 'banana': 1}
```

右侧先读取旧值，左侧再写回新值。第一次遇到单词时，`get` 提供初始值 `0`。

标准库 `collections.Counter` 直接表达“计数器”意图：

```
from collections import Counter

counts = Counter(words)
print(counts["apple"])    # 2
print(counts["missing"])  # 0
```

README README README README READM
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-6789185d35a23d3e67dca153f08.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

使用字典分组

分组算法的核心是"根据记录算出键，再把记录追加到对应列表"。普通字典可使用 `setdefault`：

```
groups: dict[str, list[str]] = {}

for word in words:
    initial = word[0]
    bucket = groups.setdefault(initial, [])
    bucket.append(word)
```

第一次遇到键时，`setdefault` 把新列表存入字典；之后返回已经存在的列表。`bucket.append` 修改的正是字典里保存的列表。

当整个算法都遵守"缺键时创建空列表"这条规则时，`defaultdict(list)` 更直接：

```
from collections import defaultdict

groups: defaultdict[str, list[str]] = defaultdict(list)
for word in words:
    groups[word[0]].append(word)
```

需要留意：读取 `groups[missing_key]` 会调用默认工厂并把新条目插入 `defaultdict`。如果只是想检查而不修改它，使用 `get` 或 `in`。偶尔才需要一个默认值时，普通 `dict` 往往更清楚。

5.12 嵌套可变默认值的共享陷阱

`dict.fromkeys(keys, value)` 会让所有键引用同一个 `value`，不会为每个键分别调用或复制它：

```
wrong = dict.fromkeys(["backend", "data"], [])
wrong["backend"].append("Python")
print(wrong)
# {'backend': ['Python'], 'data': ['Python']}
```

这相当于给同一个列表贴了两个键标签。只要默认值是可变对象，通常都不应这样初始化。可以显式地为每个键创建新列表：

```python
correct: dict[str, list[str]] = {}
for team in ["backend", "data"]:
    correct[team] = []

correct["backend"].append("Python")
print(correct)
# {'backend': ['Python'], 'data': []}
```

下一章学习字典推导式后，也可以写成 `{team: [] for team in teams}`。关键不在语法长短，而在每次循环都真正执行一次 `[]`，创建独立列表。

数据结构选择速查

需求	推荐类型或工具	不推荐的绕路
保留重复值和位置顺序	`list`	用集合后再猜原顺序
唯一成员与高频成员判断	`set`	每次在线性列表中查找
不可变的一组唯一成员	`frozenset`	把普通集合当字典键
唯一键到值的映射	`dict`	用两个平行列表维护对应关系
按出现次数统计	`Counter`	到处重复手写初始化分支
按键收集多个值	`defaultdict(list)` 或 `setdefault`	多层重复的缺键判断

常见错误速查

错误写法或假设	实际问题	建议
`empty = {}` 创建空集合	实际创建的是字典	使用 `set()`
依赖集合输出顺序	集合没有可靠业务顺序	展示时 `sorted`，业务有序时换结构
把列表放进集合或作为字典键	列表不可哈希	重新设计键，或在合适时用元组
`skills.update("SQL")`	字符串会被拆成三个字符	单项用 `add`，多项传字符串集合
把 `set.pop()` 当成删除末项	集合没有“末项”语义	删除指定值用 `remove / discard`

常见错误速查

错误写法或假设	实际问题	建议
`empty = {}` 创建空集合	实际创建的是字典	使用 `set()`
依赖集合输出顺序	集合没有可靠业务顺序	展示时 `sorted`，业务有序时换结构
把列表放进集合或作为字典键	列表不可哈希	重新设计键，或在合适时用元组
`skills.update("SQL")`	字符串会被拆成三个字符	单项用 `add`，多项传字符串集合
把 `set.pop()` 当成删除末项	集合没有“末项”语义	删除指定值用 `remove / discard`
`skills = skills.add(value)`	`add` 返回 `None`	单独调用原地方法
用 `get` 后以为默认键已插入	`get` 只读取，不修改	需要插入时明确赋值或用 `setdefault`
用 `get` 无法区分缺键与值为 `None`	两种情况都可能得到 `None`	使用 `key in mapping`
遍历字典时新增或删除键	迭代期间大小变化会报错	遍历键快照或构造新字典
`dict.fromkeys(keys, [])` 创建多份列表	所有键共享同一个列表	循环或推导式逐个创建

Java 心智切换

Java 开发者常从 `HashSet`、`HashMap` 的方法名出发思考；Python 更鼓励先辨认数据之间的关系，再直接写出集合代数、成员判断或键值访问。迁移时重点回答四个问题：

1. 数据表达位置顺序、唯一成员，还是键值关系？
2. 缺少某个成员或键是正常分支，还是应该立即报错？
3. 当前操作创建新容器，还是会让所有别名看到原地修改？
4. 作为集合元素或字典键的对象，在查找期间是否稳定可哈希？

Python 的 `set / dict` 接近 Java 的 `HashSet / HashMap`，但语法更紧凑。紧凑不是目的；让数据结构准确表达业务约束，才是这些写法更容易维护的原因。

小练习

练习 1：权限差异

2502x6dg8fpjj0sff40000gp/T/crossnote-f-6789185d35a23d3e67dca153f08.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

Java 开发者常从 `HashSet`、`HashMap` 的方法名出发思考；Python 更鼓励先辨认数据之间的关系，再直接写出集合代数、成员判断或键值访问。迁移时重点回答四个问题：

1. 数据表达位置顺序、唯一成员，还是键值关系？
2. 缺少某个成员或键是正常分支，还是应该立即报错？
3. 当前操作创建新容器，还是会让所有别名看到原地修改？
4. 作为集合元素或字典键的对象，在查找期间是否稳定可哈希？

Python 的 `set / dict` 接近 Java 的 `HashSet / HashMap`，但语法更紧凑。紧凑不是目的；让数据结构准确表达业务约束，才是这些写法更容易维护的原因。

小练习

练习 1：权限差异

给定用户已有权限和接口必需权限，输出共同权限、缺失权限、多余权限，以及是否可以访问。要求输出稳定可读，不依赖集合自身顺序。

练习 2：保序去重

给定 `['java', 'python', 'java', 'go', 'python']`，分别得到“只关心唯一值”的集合和“保留首次出现顺序”的列表。解释为什么两种结果适用于不同需求。

练习 3：配置合并

合并默认配置、环境配置和用户配置，后面的同名键覆盖前面。分别写出“保留所有输入不变”和“允许原地更新默认配置”的版本，并用 `is` 或修改后的内容验证区别。

练习 4：订单分组与计数

给定多条 `（部门，订单号）` 记录，构造 `dict[str, list[str]]`，并统计每个部门的订单数。分别尝试普通字典、`setdefault` 和 `defaultdict(list)`，比较哪种意图最清楚。

练习 5：修复共享列表

运行 `dict.fromkeys(["A", "B", "C"], [])`，向 `A` 对应列表添加一个值，观察其他键。改写为每个键拥有独立列表，并用 `is` 验证任意两个列表不是同一个对象。

```python
"""第 5 章：集合、字典、计数与分组。

这个文件把每个知识点拆成独立示例。直接运行后，可以对照输出观察：
容器如何保证唯一性、缺失键如何处理，以及一次操作是否修改原对象。
"""

from collections import Counter, defaultdict


def show_section(title: str) -> None:
    """为每组输出增加清晰边界。"""
    print(f"\n{'=' * 12} {title} {'=' * 12}")


def demonstrate_set_creation() -> None:
    """展示集合去重、空集合写法和可哈希边界。"""
    show_section("5.1~5.2 集合创建与可哈希元素")

    raw_skills: list[str] = ["Java", "Python", "Java", "SQL"]
    unique_skills: set[str] = set(raw_skills)
    print("原始列表:", raw_skills)
    print("集合去重:", sorted(unique_skills))

    empty_set: set[Any] = set()
    empty_dict: dict[Any, Any] = {}
    print("set() / {} 的类型:", type(empty_set).__name__, type(empty_dict).__name__)

    # 不可变字符串和只含可哈希元素的元组，都可以放进集合。
    service_owners: set[tuple[str, str]] = {("order-service", "Alice"), ("user-service", "Bob")}
    print("可哈希元组:", sorted(service_owners))

    # 教学反例：list 会原地改变，不能提供稳定哈希值。
    try:
        set([["Java"], ["Python"]])
    except TypeError as error:
        print("列表不能作为集合元素:", type(error).__name__, "-", error)
```

```python
def demonstrate_membership_and_ordered_deduplication() -> None:
    """展示成员判断，以及是否保留顺序时的不同去重方案。"""
    show_section("5.3 成员判断与去重")

    allowed_roles: set[str] = {"admin", "operator"}
    current_role = "operator"
    print("角色是否允许:", current_role in allowed_roles)
    print("guest 是否允许:", "guest" in allowed_roles)

    languages: list[str] = ["Java", "Python", "Java", "Go", "Python"]
    unordered_unique: set[str] = set(languages)
    ordered_unique: list[str] = list(dict.fromkeys(languages))
    print("只关心唯一值:", sorted(unordered_unique))
    print("保留首次出现顺序:", ordered_unique)


def demonstrate_set_mutation() -> None:
    """对比集合的单项/批量添加，以及严格/宽松删除。"""
    show_section("5.4 修改集合")

    skills: set[str] = {"Java"}
    skills.add("Python")  # add 把字符串作为一个完整元素。
    skills.update(["SQL", "Git"])  # update 遍历输入，逐个添加元素。
    add_result: None = skills.add("Docker")
    print("添加后的技能:", sorted(skills))
    print("add 的返回值:", add_result)

    # 教学反例：字符串可迭代，update 会把它拆成字符。
    wrong_update: set[str] = {"Java"}
    wrong_update.update("SQL")
    print("错误使用 update 添加字符串:", sorted(wrong_update))

    # 缺失是正常情况时，discard 可以直接表达“有就删除”。
    skills.discard("Rust")
    try:
        skills.remove("Rust")
```

```python
def demonstrate_set_mutation() -> None:
    try:
        skills.remove("Rust")
    except KeyError as error:
        print("remove 删除缺失元素:", type(error).__name__, "-", error)

    # pop 删除任意元素，不要把它用于依赖顺序的业务。
    # 这里使用单元素集合，让演示结果保持确定。
    temporary: set[str] = {"temporary"}
    popped: str = temporary.pop()
    print("pop 返回被删元素:", popped, "剩余:", temporary)


def demonstrate_set_algebra() -> None:
    """展示集合代数、权限关系，以及新对象和原地修改的区别。"""
    show_section("5.5 集合代数与权限关系")

    backend: set[str] = {"Java", "Python", "SQL"}
    data: set[str] = {"Python", "SQL", "Pandas"}
    print("并集:", sorted(backend | data))
    print("交集:", sorted(backend & data))
    print("仅后端:", sorted(backend - data))
    print("对称差:", sorted(backend ^ data))
    print("两组是否无交集:", backend.isdisjoint(data))

    granted: set[str] = {"order:read", "order:write", "report:read"}
    required: set[str] = {"order:read", "order:write"}
    print("是否满足必需权限:", required <= granted)
    print("缺失权限:", sorted(required - granted))

    original: set[str] = {"read"}
    alias: set[str] = original
    combined: set[str] = original | {"write"}  # 创建新集合，不影响 original 和 alias。
    print("使用 | 后的新集合:", sorted(combined))
    print(
        "使用 | 后原集合:",
```

```
 85  def demonstrate_set_algebra() -> None:
106      print(
107          "使用 | 后原集合:",
108          sorted(original),
109          "别名仍指向原集合:",
110          alias is original,
111      )
112
113      original |= {"delete"}  # 原地修改，因此 alias 也能观察到 delete。
114      print("使用 |= 后原集合 / 别名:", sorted(original), sorted(alias))
115
116
117  def demonstrate_frozenset() -> None:
118      """展示不可变集合如何作为字典键。"""
119      show_section("5.6 frozenset")
120
121      role_by_permissions: dict[frozenset[str], str] = {
122          frozenset({"read"}): "viewer",
123          frozenset({"read", "write"}): "editor",
124      }
125      current_permissions: frozenset[str] = frozenset({"write", "read"})
126      print("权限组合对应角色:", role_by_permissions[current_permissions])
127      print("frozenset 仍支持交集:", sorted(current_permissions & {"write", "delete"}))
128
129
130  def demonstrate_dict_creation_and_access() -> None:
131      """展示字典创建、覆盖、插入顺序与四种缺失键策略。"""
132      show_section("5.7~5.8 字典创建与取值")
133
134      scores: dict[str, int] = {"Alice": 95, "Bob": 88}
135      scores["Cara"] = 91
136      scores["Alice"] = 96  # 已有键只覆盖值，不会产生第二个 Alice。
137      print("成绩字典:", scores)
138      print("Alice 分数:", scores["Alice"])
139      print("键的插入顺序:", list(scores))
140      print("Alice 是键:", "Alice" in scores)
141      print("96 是键 / 是值:", 96 in scores, 96 in scores.values())
```

```
130 def demonstrate_dict_creation_and_access() -> None:
141     print("96 是键 / 是值:", 96 in scores, 96 in scores.values())
142
143     config: dict[str, int | None] = {"timeout": 30, "proxy": None}
144     print("缺失键的默认值:", config.get("retries", 3))
145     print("get 不会插入默认键:", "retries" in config)
146     print("值为 None / 缺键:", config.get("proxy"), config.get("missing_proxy"))
147     print("用 in 区分两种 None:", "proxy" in config, "missing_proxy" in config)
148
149     # 缺键代表程序或数据错误时，[] 能让问题立即暴露。
150     try:
151         print(config["missing"])
152     except KeyError as error:
153         print("方括号访问缺失键:", type(error).__name__, "-", error)
154
155
156 def demonstrate_dict_mutation_and_merging() -> None:
157     """展示字典覆盖、合并、删除、别名和浅拷贝。"""
158     show_section("5.9 修改、合并与删除字典")
159
160     defaults: dict[str, Any] = {"theme": "light", "page_size": 20}
161     user_settings: dict[str, str] = {"theme": "dark"}
162     effective: dict[str, Any | str] = defaults | user_settings  # 新字典中由右侧同名键获胜。
163     print("合并后的有效配置:", effective)
164     print("使用 | 后默认配置不变:", defaults)
165
166     defaults.update({"language": "zh-CN"})
167     defaults |= {"page_size": 50}
168     print("update / |= 原地修改:", defaults)
169
170     status: dict[str, str] = {"status": "pending"}
171     status_alias: dict[str, str] = status
172     status_copy: dict[str, str] = status.copy()
173     status["status"] = "paid"
174     print("别名看到外层修改:", status_alias["status"])
175     print("浅拷贝保留旧状态:", status_copy["status"])
```

```python
def demonstrate_dict_mutation_and_merging() -> None:

    nested: dict[str, list[str]] = {"events": ["created"]}
    nested_copy: dict[str, list[str]] = nested.copy()  # 只复制外层字典，内部 events 列表仍共享。
    nested["events"].append("paid")
    print(
        "浅拷贝仍共享内层列表:",
        nested_copy["events"],
        nested_copy["events"] is nested["events"],
    )

    removable: dict[str, int] = {"first": 1, "second": 2, "third": 3}
    removed_value: int = removable.pop("second")
    missing_value: int = removable.pop("missing", 0)
    last_pair: tuple[str, int] = removable.popitem()
    print("pop 的值 / 缺键默认值:", removed_value, missing_value)
    print("popitem 的最后插入项:", last_pair, "剩余:", removable)


def demonstrate_dict_views_and_safe_iteration() -> None:
    """展示动态视图，以及遍历时修改字典大小的错误与修复。"""
    show_section("5.10 字典视图与安全遍历")

    scores: dict[str, int] = {"Alice": 95, "Bob": 88}
    names_view: dict_keys = scores.keys()
    print("添加前的动态键视图:", list(names_view))
    scores["Cara"] = 91
    print("添加后的同一个键视图:", list(names_view))

    print("使用 items 遍历:")
    for name, score in scores.items():
        print(f"  {name} -> {score}")

    unsafe_scores: dict[str, int] = {"Alice": 95, "Bob": 88, "Cara": 91}
    try:
        for name in unsafe_scores:
            if unsafe_scores[name] < 90:
```

```
194 def demonstrate_dict_views_and_safe_iteration() -> None:
209     try:
210         for name in unsafe_scores:
211             if unsafe_scores[name] < 90:
212                 del unsafe_scores[name]
213     except RuntimeError as error:
214         print("遍历时改变字典大小:", type(error).__name__, "-", error)
215
216     safe_scores: dict[str, int] = {"Alice": 95, "Bob": 88, "Cara": 91}
217     for name in list(safe_scores):  # list 创建键快照，原字典可安全删除。
218         if safe_scores[name] < 90:
219             del safe_scores[name]
220     print("通过键快照安全删除:", safe_scores)
221
222
223 def demonstrate_counting_and_grouping() -> None:
224     """用普通字典、Counter、setdefault 和 defaultdict 完成聚合。"""
225     show_section("5.11 计数与分组")
226
227     words: list[str] = ["apple", "apricot", "banana", "blueberry", "apple"]
228
229     manual_counts: dict[str, int] = {}
230     for word in words:
231         # 右侧先取得旧次数，第一次出现时从 0 开始，
232         # 再把结果写回同一个键。
233         manual_counts[word] = manual_counts.get(word, 0) + 1
234     print("普通字典计数:", manual_counts)
235
236     counts: Counter[str] = Counter(words)
237     print("Counter 计数:", dict(counts))
238     print("Counter 的缺失项:", counts["missing"])
239
240     setdefault_groups: dict[str, list[str]] = {}
241     for word in words:
242         initial: str = word[0]
243         bucket: list[str] = setdefault_groups.setdefault(initial, [])
```

```
241     for word in words:
242         initial: str = word[0]
243         bucket: list[str] = setdefault_groups.setdefault(initial, [])
244         bucket.append(word)  # bucket 就是字典中保存的那个列表。
245     print("setdefault 分组:", setdefault_groups)
246
247     grouped: defaultdict[str, list[str]] = defaultdict(list)
248     for word in words:
249         grouped[word[0]].append(word)
250     print("defaultdict 分组:", dict(grouped))
251
252     # defaultdict 的 [] 访问会创建缺失项；get 只读取，不会触发默认工厂。
253     print("get 读取缺失组:", grouped.get("z"), "z 是否已插入:", "z" in grouped)
254     empty_group: list[str] = grouped["z"]
255     print("[] 读取缺失组:", empty_group, "z 是否已插入:", "z" in grouped)
256
257
258 def demonstrate_shared_mutable_defaults() -> None:
259     """让 dict.fromkeys 共享同一个可变值的问题变得可观察。"""
260     show_section("5.12 嵌套可变默认值")
261
262     # 教学反例：两个键都引用传给 fromkeys 的同一个列表。
263     shared: dict[str, list[Any]] = dict.fromkeys(["backend", "data"], [])
264     shared["backend"].append("Python")
265     print("fromkeys 共享列表:", shared)
266     print("两个值是否同一对象:", shared["backend"] is shared["data"])
267
268     # 正确示范：每次循环都执行一次 []，为每个键创建独立列表。
269     independent: dict[str, list[str]] = {}
270     for team in ["backend", "data"]:
271         independent[team] = []
272     independent["backend"].append("Python")
273     print("逐个创建独立列表:", independent)
274     print("两个值是否同一对象:", independent["backend"] is independent["data"])
275
```

```python
def demonstrate_readme_exercises() -> None:
    """运行 README 中的权限、去重、配置、分组和共享状态练习。"""
    show_section("README 小练习")

    print("练习 1: 权限差异")
    granted: set[str] = {"order:read", "report:read", "profile:edit"}
    required: set[str] = {"order:read", "order:write"}
    print("  共同权限:", sorted(granted & required))
    print("  缺失权限:", sorted(required - granted))
    print("  多余权限:", sorted(granted - required))
    print("  是否可以访问:", required <= granted)

    print("练习 2: 保序去重")
    languages: list[str] = ["java", "python", "java", "go", "python"]
    unique_languages: set[str] = set(languages)
    ordered_languages: list[str] = list(dict.fromkeys(languages))
    print("  只关心唯一值:", sorted(unique_languages))
    print("  保留首次出现顺序:", ordered_languages)

    print("练习 3: 配置合并")
    defaults: dict[str, Any] = {"theme": "light", "page_size": 20, "debug": False}
    environment: dict[str, Any] = {"debug": True, "region": "cn-east"}
    user: dict[str, Any] = {"theme": "dark", "page_size": 50}
    effective: dict[str, Any] = defaults | environment | user
    print("  创建新配置:", effective)
    print("  三个输入保持不变:", defaults, environment, user)

    mutable_defaults: dict[str, Any] = defaults.copy()
    defaults_alias: dict[str, Any] = mutable_defaults
    mutable_defaults.update(environment)
    mutable_defaults.update(user)
    print("  原地合并结果:", mutable_defaults)
    print("  别名仍是同一个字典:", defaults_alias is mutable_defaults)

    print("练习 4: 订单分组与计数")
    orders: list[tuple[str, str]] = [
```

```
277 def demonstrate_readme_exercises() -> None:
311     print("练习 4: 订单分组与计数")
312     orders: list[tuple[str, str]] = [
313         ("backend", "O-101"),
314         ("data", "O-102"),
315         ("backend", "O-103"),
316         ("frontend", "O-104"),
317     ]
318
319     plain_groups: dict[str, list[str]] = {}
320     for department, order_id in orders:
321         if department not in plain_groups:
322             plain_groups[department] = []
323         plain_groups[department].append(order_id)
324
325     setdefault_groups: dict[str, list[str]] = {}
326     for department, order_id in orders:
327         setdefault_groups.setdefault(department, []).append(order_id)
328
329     default_groups: defaultdict[str, list[str]] = defaultdict(list)
330     for department, order_id in orders:
331         default_groups[department].append(order_id)
332
333     order_counts: dict[str, int] = {department: len(order_ids) for department, order_ids in plain_groups.items()}
334     print("  普通字典:", plain_groups)
335     print("  setdefault:", setdefault_groups)
336     print("  defaultdict:", dict(default_groups))
337     print("  各部门订单数:", order_counts)
338
339     print("练习 5: 修复共享列表")
340     shared: dict[str, list[Any]] = dict.fromkeys(["A", "B", "C"], [])
341     shared["A"].append("value")
342     print("  fromkeys 的联动结果:", shared)
343     print("  A/B 共享对象:", shared["A"] is shared["B"])
344
345     independent: dict[str, list[Any]] = {key: [] for key in ("A", "B", "C")}
```

```
277  def demonstrate_readme_exercises() -> None:
344
345      independent: dict[str, list[Any]] = {key: [] for key in ("A", "B", "C")}
346      independent["A"].append("value")
347      print("  独立列表结果:", independent)
348      print("  A/B 是不同对象:", independent["A"] is not independent["B"])
349
350
351  def main() -> None:
352      demonstrate_set_creation()
353      demonstrate_membership_and_ordered_deduplication()
354      demonstrate_set_mutation()
355      demonstrate_set_algebra()
356      demonstrate_frozenset()
357      demonstrate_dict_creation_and_access()
358      demonstrate_dict_mutation_and_merging()
359      demonstrate_dict_views_and_safe_iteration()
360      demonstrate_counting_and_grouping()
361      demonstrate_shared_mutable_defaults()
362      demonstrate_readme_exercises()
363
364
365  if __name__ == "__main__":
366      main()
367
```